

Chapter 59: Performance Issues

Scenario 1: The 50-Widget Dashboard That Jitters on Every Tick

Situation: A finance dashboard renders 50 independent widgets (charts, KPI tiles, tables). A single WebSocket pushes a price tick every 500ms for one symbol. Chrome DevTools Performance panel shows a 45ms "Recalculate Style + Layout + Paint" block on every tick, and the frame rate drops to ~18fps even though only one widget's data actually changed.

Question: How do you diagnose why all 50 widgets re-render when only one data point changed, and how do you fix it?

Answer:

Diagnosis: Open the Performance tab, record 5 seconds of ticks, and look at the "Timings" and "Main" flame chart. If you see `ApplicationRef.tick` firing and then dozens of component `refreshView` calls in the same frame, Angular's default (Zone-based) change detection is walking the *entire* component tree on every async event — because `zone.js` patches the WebSocket `onmessage` callback and triggers global change detection.

Root cause: default `ChangeDetectionStrategy.Default` means every component re-checks its bindings on every CD cycle, even components with `@Input()` s that are reference-identical. With 50 widgets that's 50+ re-renders for a 1-widget change.

Fix — two layers:

1. Put every widget on `OnPush` so Angular only checks it when its `@Input()` reference changes, an event originates inside it, or an `async` /signal source it reads emits.
2. Model the price feed as a signal per symbol (or a keyed signal map) so only the widget bound to that symbol's signal recomputes — no `markForCheck` broadcast to siblings.

```
@Component({
  selector: 'app-price-widget',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div class="widget">
      <span>{{ symbol }}</span>
      <span>{{ price() | currency }}</span>
    </div>
  `,
})
export class PriceWidgetComponent {
  @Input({ required: true }) symbol!: string;
  private priceService = inject(PriceService);
  price = computed(() => this.priceService.prices()[this.symbol]);
}

@Injectable({ providedIn: 'root' })
export class PriceService {
  private _prices = signal<Record<string, number>>({});
  prices = this._prices.asReadonly();
}
```

```

updatePrice(symbol: string, value: number) {
  this._prices.update(p => ({ ...p, [symbol]: value }));
}
}

```

Because `computed()` only notifies consumers of the specific symbol slice they read, and each widget is `OnPush` with a signal read directly in the template, Angular's fine-grained reactivity (zoneless-ready) skips the other 49 widgets entirely.

Measurement: re-run the Performance recording — the "Recalculate Style" block should shrink from ~45ms/tick to sub-5ms, and only one widget's DOM subtree should highlight in the "Rendering > Paint flashing" overlay instead of all 50.

Interviewer intent: Tests whether the candidate understands that Zone.js change detection is tree-wide by default and that `OnPush` + signals is the modern way to scope re-renders, not just "add `trackBy`."

Scenario 2: 10,000-Row Table That Scrolls Like Molasses

Situation: An admin table renders 10,000 rows with `@for`. Scrolling causes visible jank (Chrome's FPS meter shows 8-12fps), and the DOM inspector shows 10,000 `<tr>` elements mounted at once. Initial render also blocks the main thread for 1.2 seconds.

Question: What's the right strategy to make this scroll smoothly, and how would you implement it?

Answer:

Diagnosis: The Elements panel confirms all 10,000 rows exist in the DOM simultaneously — layout and paint cost scale with total DOM node count, not visible node count. The Performance panel shows long "Layout" tasks during scroll because the browser recalculates layout for thousands of offscreen nodes whenever any style changes (e.g., hover states, sticky headers).

Root cause: rendering the entire dataset instead of only the viewport window. `@for` alone doesn't virtualize — it just efficiently diffs whatever list you give it.

Fix: Use the CDK Virtual Scroll (`@angular/cdk/scrolling`), which only mounts the rows currently in/near the viewport (typically 15-30 DOM nodes regardless of dataset size).

```

import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  selector: 'app-user-table',
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="48" class="viewport">
      <div *cdkVirtualFor="let user of users(); trackBy: trackByUserId" class="row">
        {{ user.name }} – {{ user.email }}
      </div>
    </cdk-virtual-scroll-viewport>
  `,
  styles: [`.viewport { height: 600px; }`],
  changeDetection: ChangeDetectionStrategy.OnPush,
})

```

```

})
export class UserTableComponent {
  users = input.required<User[]>();
  trackByUserId = (_, u: User) => u.id;
}

```

For the initial-render cost, additionally defer the table's construction until it's actually needed (e.g., behind a tab) using `@defer (on viewport)`, and paginate/virtualize the underlying data fetch so you're not deserializing 10,000 JSON records upfront if avoidable.

Measurement: DOM node count during scroll should stay flat (~30 rows) regardless of dataset size in the Elements panel; the Performance recording should show scroll frames at 55-60fps; Lighthouse's "Avoid excessive DOM size" audit should stop flagging the page.

Interviewer intent: Confirms the candidate knows DOM size — not just JS execution — drives layout/paint cost, and that virtual scrolling (not just `trackBy`) is the fix for large lists.

Scenario 3: Initial Load Scores 38 on Lighthouse Performance

Situation: Lighthouse reports Performance: 38, with "Reduce unused JavaScript" flagging 1.4MB of unused bytes, a Total Blocking Time of 2,300ms, and a main bundle (`main.js`) of 3.8MB. The app is a single large standalone bootstrap with everything imported eagerly, including a charting library, a rich text editor, and `moment.js` with all locales.

Question: Walk through how you'd cut the bundle size and improve TBT/LCP.

Answer:

Diagnosis: Run `ng build --stats-json` and open the output with `webpack-bundle-analyzer` (or Angular's `source-map-explorer`) to see which modules dominate. Typically you'll find: (1) heavy libraries loaded on the initial chunk that are only used on secondary routes, (2) a locale-heavy date library instead of the native `Intl` API, (3) no route-level code splitting.

Fixes, in priority order:

1. **Route-level lazy loading** with standalone routes so non-landing-page code isn't in the initial chunk:

```

export const routes: Routes = [
  { path: '', component: DashboardComponent },
  {
    path: 'reports',
    loadComponent: () => import('./reports/reports.component')
      .then(m => m.ReportsComponent),
  },
];

```

2. **`@defer` for heavy, below-the-fold, or interaction-gated UI** (charts, editors) so they don't count toward the critical bundle at all:

```
@defer (on viewport; prefetch on idle) {
  <app-rich-chart [data]="chartData()" />
} @placeholder (minimum 100ms) {
  <div class="chart-skeleton"></div>
} @loading (after 100ms) {
  <app-spinner />
}
```

3. **Replace moment.js** with `date-fns` (tree-shakeable, import only what you use) or native `Intl.DateTimeFormat`, eliminating hundreds of KB of locale data.
4. **Verify budgets** in `angular.json` so regressions fail CI:

```
"budgets": [
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" }
]
```

Measurement: Re-run `ng build` and compare initial chunk size (target well under 1MB), then re-run Lighthouse — expect TBT to drop under 300ms and Performance score to climb into the 80-90s. Confirm with the Network panel that `reports` and chart code only load when that route/section is actually visited.

Interviewer intent: Tests real-world bundle-analysis workflow and knowledge of `@defer` /lazy routes as the modern replacement for manual `NgModule` code-splitting tricks.

Scenario 4: Search Box Lags Behind Keystrokes

Situation: A product search input filters a 5,000-item in-memory list on every keystroke. Users report visible lag — each keypress takes ~180ms to reflect in the UI, and typing "wireless mouse" feels like the input is fighting back.

Question: How do you find and fix the source of the input lag?

Answer:

Diagnosis: Record a Performance trace while typing. Each keystroke shows a long task: `(anonymous) → filterList → ChangeDetection` taking ~150-180ms, meaning the filter (and the subsequent OnPush-unfriendly re-render of the whole list) runs synchronously on every single keystroke, and — critically — on every keystroke the *entire* 5,000-item array is filtered and the DOM diffed, blocking the next paint.

Root cause: no debouncing on the input, combined with an expensive synchronous filter and a component that isn't OnPush, so a keystroke also triggers unrelated sibling re-checks.

Fix: debounce the input signal with RxJS (via `toSignal` / `toObservable` bridge) or a simple timer-based signal effect, `onPush` the list component, and if filtering itself is slow, filter with `computed()` so it's memoized against the same query.

```
@Component({
  selector: 'app-product-search',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <input (input)="query.set($any($event.target).value)" placeholder="Search..." />
```

```

    @for (item of filtered(); track item.id) {
      <app-product-row [product]="item" />
    }
  },
})
export class ProductSearchComponent {
  private allProducts = inject(ProductStore).products;
  query = signal('');

  private debouncedQuery = toSignal(
    toObservable(this.query).pipe(debounceTime(200), distinctUntilChanged()),
    { initialValue: '' },
  );

  filtered = computed(() => {
    const q = this.debouncedQuery().toLowerCase();
    if (!q) return this.allProducts();
    return this.allProducts().filter(p => p.name.toLowerCase().includes(q));
  });
}

```

If 200ms debounce still isn't enough because the filter itself is heavy (e.g., fuzzy search), move the filter to a Web Worker via Angular CLI's worker support (`ng generate web-worker`) so the main thread stays free for input handling entirely.

Measurement: Re-record the Performance trace — keystroke-to-paint tasks should drop under 16ms (the input update is now uncoupled from filtering), and the "Input Delay" section of the trace's Interactions track should show negligible values. Confirm via the "Total Blocking Time" in a Lighthouse run against an interaction-heavy scenario, or via `event timing` API in production RUM.

Interviewer intent: Checks whether the candidate reaches for debounce + memoization + OnPush together instead of just slapping a `debounceTime` on and calling it done.

Scenario 5: A Chart Re-Renders on Every Unrelated State Change

Situation: A dashboard has a `<app-revenue-chart>` component and a `<app-notifications-bell>` component that polls unread counts every 5 seconds. Every poll, the chart also flashes/re-renders (visible via "Paint flashing" in DevTools), even though its own data hasn't changed. The chart render is expensive (~60ms).

Question: Why does an unrelated component's polling cause the chart to redraw, and how do you stop it?

Answer:

Diagnosis: Enable "Paint flashing" in DevTools Rendering tab — the chart's bounding box flashes green every 5 seconds in sync with the notification poll, even though no input to the chart changed. Checking the component tree, both live under a shared parent that's still `ChangeDetectionStrategy.Default`, or the chart is `OnPush` but its `[data]` input is a *new array/object reference* created on every parent re-render (e.g., `[data]="computeChartData()"` called inline in the template, recreating a new array each CD pass).

Root cause: either (a) a non-OnPush ancestor triggers global CD which re-checks the chart's template even though OnPush should have skipped it — usually because the chart also isn't OnPush — or (b) a fresh object/array reference defeats OnPush's reference-equality input check, or (c) the chart internally reads a signal/service value that changes as a side effect of the poll (e.g., a shared "lastUpdated" timestamp field on the same store object).

Fix — apply all three defenses:

```
@Component({
  selector: 'app-revenue-chart',
  changeDetection: ChangeDetectionStrategy.OnPush, // (1) opt in to reference checks
  template: `<canvas #chartCanvas></canvas>`,
})
export class RevenueChartComponent {
  // (2) memoize so the array reference is stable across unrelated polls
  data = input.required<ChartPoint[]>();
}
```

```
// Parent – do NOT call a function inline in the template; use a computed()
chartData = computed(() => this.revenueStore.points()); // stable ref unless points()
actually changes
```

Also split the store so notification count and revenue data are independent signals rather than fields on one shared object — reading `store.state().unreadCount` inside a component that ignores `revenueData` still re-triggers if `state()` is a single signal wrapping both, because any field change produces a new object and any `computed()` /effect depending on `state()` reruns.

```
// Bad: one signal, any change reruns everything reading it
state = signal({ unreadCount: 0, revenueData: [] });

// Good: independent signals
unreadCount = signal(0);
revenueData = signal<ChartPoint[]>([]);
```

Measurement: With Paint flashing back on, the chart's box should stay dark during notification polls and only flash when `revenueData` truly changes. Confirm with the Performance panel that the 60ms chart-render task no longer appears every 5 seconds.

Interviewer intent: Digs into the difference between "component is OnPush" and "component actually skips re-render" — testing understanding of reference identity and signal granularity, a very common real bug.

Scenario 6: Route Transitions Feel Sluggish

Situation: Clicking between "Dashboard" and "Settings" tabs shows a visible ~800ms white flash before content appears, even though both routes are lazy-loaded and the network tab shows the chunk downloads in 40ms (already cached).

Question: If the JS is already cached and loads fast, what's causing the 800ms delay, and how do you fix it?

Answer:

Diagnosis: The Network tab confirms the chunk is served from cache almost instantly, so the delay must be in JS execution or change detection, not network. Performance trace during the click shows a long task labeled with the target component's constructor/`ngOnInit`, often containing synchronous work: a big `computed` recalculation, a blocking `JSON.parse` of cached data, or — very commonly — an `HttpClient` call in a route `resolver` /guard that blocks navigation until it resolves, followed by CD over a huge freshly-instantiated tree with `Default` strategy.

Root cause candidates to check in order: (1) a route resolver awaiting a slow API call before the route even activates (navigation literally waits); (2) heavy synchronous initialization logic in the new component's constructor; (3) the outgoing view not being destroyed efficiently, so both trees briefly coexist.

Fix depends on root cause. If it's a resolver blocking navigation for non-critical data:

```
// Bad: navigation blocked until settings fully loaded
{ path: 'settings', loadComponent: ..., resolve: { settings: settingsResolver } }

// Better: navigate immediately, stream data in with a loading state
@Component({ ... changeDetection: ChangeDetectionStrategy.OnPush })
export class SettingsComponent {
  private settingsService = inject(SettingsService);
  settings = toSignal(this.settingsService.getSettings(), { initialValue: null });
}
```

```
@if (settings(); as s) {
  <app-settings-form [settings]="s" />
} @else {
  <app-settings-skeleton />
}
```

Also add a lightweight route-transition placeholder so the perceived latency drops even if actual work stays similar, and consider `withComponentInputBinding()` plus prefetching the next likely route's chunk via `@defer` (prefetch on hover) or the Router's preloading strategy (`PreloadAllModules` or a custom quicklink-style strategy) so the *first* click after app load isn't the one paying the cold-fetch cost.

Measurement: Time from click to first paint of new content, measured via `performance.mark / measure` around `Router.events` (`NavigationStart` → post-render `NgZone.onStable` or an `afterNextRender` hook), should drop from ~800ms to under 100ms; confirm visually that no blocking spinner-free white flash remains.

Interviewer intent: Tests whether the candidate distinguishes network latency from render/execution latency and knows resolvers can block navigation.

Scenario 7: Third-Party Analytics Script Blocks the Main Thread

Situation: Lighthouse flags "Reduce the impact of third-party code," showing a 620ms main-thread block attributed to an analytics vendor's script loaded via a `<script>` tag in `index.html`. Users on mid-tier Android devices report the page feeling frozen for the first second.

Question: How do you keep third-party analytics from blocking initial interactivity?

Answer:

Diagnosis: The Performance panel's "Main" track shows a long yellow (scripting) block right at page load attributed to the vendor's domain, overlapping with First Input Delay measurements. The Network panel shows the script loaded synchronously and render-blocking (no `async` / `defer`), competing with Angular's own bootstrap for the main thread.

Root cause: a synchronous, render-blocking third-party `<script>` in the `<head>`, executed before Angular even bootstraps, competing for the single main thread during the most latency-sensitive window (LCP/TTI).

Fix, layered:

1. Load the script `async` / `defer` (or dynamically) so it doesn't block parsing/bootstrap:

```
<script src="https://analytics.vendor.com/tag.js" async></script>
```

2. Defer *initialization* until the app is idle/interactive using `requestIdleCallback` or Angular's `afterNextRender` / an app-level idle hook, so analytics never competes with the critical rendering path:

```
@Component({ selector: 'app-root', standalone: true, template: `<router-outlet />` })
export class AppComponent {
  constructor() {
    afterNextRender(() => {
      if ('requestIdleCallback' in window) {
        (window as any).requestIdleCallback(() => loadAnalytics());
      } else {
        setTimeout(() => loadAnalytics(), 3000);
      }
    });
  }
}

function loadAnalytics() {
  const s = document.createElement('script');
  s.src = 'https://analytics.vendor.com/tag.js';
  s.async = true;
  document.body.appendChild(s);
}
```

3. If the vendor supports it, self-host or use a lightweight proxy/facade pattern (only load the full SDK on first genuine interaction, e.g., first scroll) — the same "facade" technique used for embedding YouTube players.

Measurement: Re-run Lighthouse — "Reduce impact of third-party code" warning should disappear or shrink drastically, Total Blocking Time should drop, and Time to Interactive should improve by roughly the size of the blocked window (600ms+). Confirm on a throttled mid-tier device profile (4x CPU slowdown) in DevTools.

Interviewer intent: Tests awareness that not all performance problems are Angular's fault, and knowledge of `async`/`defer`/idle-loading strategies for third-party scripts.

Scenario 8: Product Images Cause Layout Shift (CLS 0.42)

Situation: Lighthouse reports a Cumulative Layout Shift of 0.42 (poor) on a product grid. Visually, as images load, cards visibly jump and push content down, and users complain about mis-clicks ("I meant to click 'Add to Cart' but the page shifted and I clicked 'Delete' on the row below").

Question: What's causing the layout shift, and how do you eliminate it using modern Angular tooling?

Answer:

Diagnosis: Lighthouse's CLS breakdown (or the Performance panel's "Experience" section, which highlights individual "Layout Shift" clusters with red rectangles) points to `<img>` tags without explicit dimensions.

Because the images load asynchronously and the browser doesn't reserve space in advance, each image's decode/load event pushes surrounding content.

Root cause: raw `` tags with no `width/height` (or matching CSS aspect-ratio box), so the browser can't reserve layout space before the image downloads, especially over slower connections where the shift is very visible.

Fix: use `NgOptimizedImage` (`provideImage/ NgOptimizedImage` directive), which enforces explicit `width/height` (or `fill` mode with a sized container) at compile time, and adds automatic `fetchpriority`, lazy loading below the fold, and a warning if you forget dimensions.

```
import { NgOptimizedImage } from '@angular/common';

@Component({
  selector: 'app-product-card',
  standalone: true,
  imports: [NgOptimizedImage],
  template: `
    <img
      [ngSrc]="product().imageUrl"
      width="400"
      height="300"
      [priority]="isAboveFold()"
      alt="{{ product().name }}"
    />
  `,
})
export class ProductCardComponent {
  product = input.required<Product>();
  isAboveFold = input(false);
}
```

For images whose final size varies (responsive grids), pair `width/height` (which set the *intrinsic aspect ratio*) with CSS that scales via `max-width: 100%; height: auto;` so the reserved space scales proportionally instead of using a fixed box that doesn't match the rendered size.

Also mark the first 2-3 above-the-fold hero/product images with `priority` (maps to `fetchpriority="high"` and preloading) to improve LCP simultaneously, since CLS and LCP fixes on hero images often go together.

Measurement: Re-run Lighthouse — CLS should drop to well under 0.1 (good), and the Performance panel's layout-shift clusters should disappear. Visually confirm no card jump occurs on a throttled 3G profile.

Interviewer intent: Tests concrete knowledge of `NgOptimizedImage` as the modern, compiler-enforced fix for CLS rather than ad-hoc CSS aspect-ratio hacks.

Scenario 9: The App Slows to a Crawl After 30 Minutes of Use

Situation: A single-page trading app runs fine at launch but after ~30 minutes of continuous use (users leave it open all day), Chrome's Task Manager shows memory climbing from 180MB to 1.4GB, and the tab eventually becomes unresponsive or crashes with "Aw, Snap!"

Question: How do you find and fix a memory leak like this in an Angular app?

Answer:

Diagnosis: Use Chrome DevTools Memory panel — take a heap snapshot at launch, use the app for a few minutes performing a repeatable action (e.g., navigate to a symbol's detail page and back, 20 times), take another snapshot, and use the "Comparison" view. Look for detached DOM trees (nodes with no document but still retained) and growing counts of listener/subscription-related constructor names.

Common root causes in Angular apps, all confirmed via the retainer tree in the snapshot:

1. RxJS subscriptions in components that are never unsubscribed (e.g., subscribing to a shared/long-lived service's Observable in `ngOnInit` without a teardown), so each component instantiation adds a permanent subscriber closure holding the whole component (and its DOM) alive.
2. Native event listeners added via `document.addEventListener` or `window.addEventListener` in a component without removal in `ngOnDestroy`.
3. A WebSocket or `setInterval` reconnect/polling loop that isn't cleared when navigating away.
4. Signals-based `effect()`s created inside a component but not scoped to that component's injection context (rare with Angular's automatic cleanup, but happens if created outside DI or with `{ manualCleanup: true }` and never destroyed manually).

Fix — the systematic remedy is to always use teardown-safe patterns:

```
@Component({ selector: 'app-symbol-detail', changeDetection: ChangeDetectionStrategy.OnPush })
export class SymbolDetailComponent implements OnDestroy {
  private destroyRef = inject(DestroyRef);
  private priceService = inject(PriceService);

  price = toSignal(
    this.priceService.priceUpdates$.pipe(takeUntilDestroyed()), // auto-unsubscribes on
    { initialValue: null },
  );

  constructor() {
    const handler = () => this.calculateLayout();
    window.addEventListener('resize', handler);
    this.destroyRef.onDestroy(() => window.removeEventListener('resize', handler));
  }
}
```

```
private recalculateLayout() { /* ... */ }
ngOnDestroy(): void { /* subscriptions already handled by takeUntilDestroyed */ }
}
```

For polling/interval-based reconnect logic, guard it the same way:

```
constructor() {
  const id = setInterval(() => this.pollStatus(), 5000);
  inject(DestroyRef).onDestroy(() => clearInterval(id));
}
```

Also audit any app-wide singleton service that accumulates entries in a `Map` /array (e.g., a cache that never evicts) — this shows up as steady growth in the snapshot's "Objects" list even with subscriptions fixed, so add an LRU eviction or TTL.

Measurement: Repeat the snapshot-diff cycle (navigate in/out 20x, snapshot, compare) — detached node counts and retained closure counts for the fixed classes should return to near-zero between cycles, and Task Manager memory should plateau instead of climbing linearly over a multi-hour soak test.

Interviewer intent: Tests systematic memory-leak diagnosis methodology (heap snapshot diffing) plus knowledge of `takeUntilDestroyed` / `DestroyRef` as the modern cleanup idiom.

Scenario 10: `*ngFor` List Re-Creates All DOM Nodes on Every Update

Situation: A live order-book list updates every 200ms with the top 20 orders (same order IDs, updated prices). Even though the list content is largely the same 20 items, DevTools shows all 20 `<tr>` elements being destroyed and recreated on every update (visible via flashing highlight and via `ngOnDestroy` / `ngOnInit` logs firing every cycle), causing scroll position to reset and hover states to flicker.

Question: Why does the list fully re-render when the underlying data is mostly unchanged, and how do you fix it?

Answer:

Diagnosis: Adding `console.log` in the row component's constructor and `ngOnDestroy` confirms new instances are created every update cycle. Inspecting the parent template shows `@for (order of orders(); track $index)` (or the legacy `*ngFor` without `trackBy`) — tracking by index means Angular treats "item at position 0" as identity, but since the *array reference* itself is replaced wholesale each poll (a new array from the API), and tracking is by index not by stable ID, Angular can't tell that "order at index 3" is the same logical order as before, especially once the order-book order shifts as prices change and orders re-sort.

Root cause: missing (or wrong) track expression — tracking by `$index` instead of a stable business key means any reordering or array-reference replacement is treated as "everything changed," destroying and recreating rows instead of patching them in place.

Fix: track by the stable order ID.

```
@Component({
  selector: 'app-order-book',
  changeDetection: ChangeDetectionStrategy.OnPush,
```

```

template: `
  @for (order of orders(); track order.id) {
    <tr class="order-row">
      <td>{{ order.id }}</td>
      <td>{{ order.price | currency }}</td>
      <td>{{ order.size }}</td>
    </tr>
  }
`,
})
export class OrderBookComponent {
  orders = input.required<Order[]>();
}

```

With `track order.id`, Angular's list-diffing algorithm matches existing DOM nodes to the same logical entity across updates, only patching the `price`/`size` bindings on existing `<tr>` elements and reordering DOM nodes (cheap) instead of destroy+recreate (expensive) when only values or positions change.

Measurement: Re-check the constructor/`ngOnDestroy` logs — they should only fire when an order genuinely enters/leaves the top 20, not every 200ms tick. Confirm scroll position and `:hover` state survive updates, and re-record the Performance trace to see per-tick DOM mutation counts drop from ~20 creates to a handful of attribute patches.

Interviewer intent: Directly tests understanding of the `track` expression in `@for` as the single most common source of unnecessary DOM churn in list-heavy UIs.

Scenario 11: A Form with 40 Fields Lags on Every Keystroke

Situation: A large settings form (40+ reactive form controls, several with cross-field validators) shows visible input lag — typing in one text field causes a ~90ms delay before the character appears, worse than the 40 fields would suggest.

Question: How would you track down and resolve form-related input lag?

Answer:

Diagnosis: Record a Performance trace while typing a single character. The flame chart shows `FormGroup.updateValueAndValidity` cascading through many sibling controls, plus the whole form's status (`valid`/`invalid`) being recomputed, triggering CD across every field component (each shows a red "validate" self-time block) even though only one control's value changed.

Root cause: (1) cross-field validators registered at the `FormGroup` level re-run for *all* controls whenever *any* control changes (this is correct Reactive Forms behavior but expensive if validators do heavy work, e.g., synchronous regex over large strings or a full-form recalculation); (2) the form's parent component (or each field wrapper) isn't `OnPush`, so every keystroke triggers a full CD sweep across all 40 rendered field components, each re-evaluating template expressions and bindings.

Fix — combine targeted validator scoping with `OnPush` and update-strategy tuning:

```
// 1. Use updateOn: 'blur' for fields where "as you type" validation isn't essential
this.form = this.fb.group({
  companyName: ['', { validators: [Validators.required], updateOn: 'blur' }],
  email: ['', { validators: [Validators.required, Validators.email] }], // stays 'change' –
  // needs live feedback
  // ...
});

// 2. Scope expensive cross-field validation to only the controls it needs,
// not the whole FormGroup, and debounce it.
this.form.get('startDate')!.valueChanges.pipe(
  debounceTime(150),
  takeUntilDestroyed(this.destroyRef),
).subscribe(() => this.validateDateRange());
```

```
@Component({
  selector: 'app-settings-field',
  changeDetection: ChangeDetectionStrategy.OnPush, // (3) each field only re-renders on its
  // own changes
  template: `...`,
})
export class SettingsFieldComponent {
  control = input.required<FormControl>();
}
```

For genuinely heavy validators (e.g., checking uniqueness against a large in-memory list), move them to `AsyncValidator` with debouncing so they don't run synchronously on the hot path of every keystroke.

Measurement: Re-trace typing in a single field — the flame chart should show only that field's validator and a small, bounded CD pass, not a 40-field cascade. Input-to-paint latency should fall from ~90ms to under 16ms, confirmed via the trace's "Interactions" lane.

Interviewer intent: Tests whether the candidate understands Reactive Forms' validation propagation model and knows `updateOn`, debounced async validators, and `OnPush` field components as levers, not just "the form is slow, add `OnPush` everywhere."

Scenario 12: Infinite Scroll Feed Leaks Memory and Slows Down Over Time

Situation: A social-feed-style page appends 20 new post cards every time the user scrolls near the bottom. After scrolling through ~2,000 posts (100 batches), scrolling becomes jittery and memory usage triples, even though old posts are no longer visible.

Question: What's the performance issue with unbounded infinite scroll, and how do you fix it while keeping the "keep scrolling" UX?

Answer:

Diagnosis: The Elements panel shows 2,000 post-card DOM subtrees still mounted, most far offscreen. The Memory panel's snapshot comparison shows growth proportional to posts loaded, and each post card likely contains an image, some event bindings, and possibly its own subscription (e.g., a "time ago" ticking timestamp).

updated every second across all 2,000 cards) — meaning that even offscreen, unneeded work keeps running (2,000 timers firing every second is itself a measurable CPU cost, visible as a recurring small task cluster in the Performance trace's "Timers" section).

Root cause: infinite scroll implemented as "keep appending, never remove," combined with per-card background work (timers, IntersectionObservers, etc.) that isn't paused/destroyed for offscreen items.

Fix: combine CDK virtual scrolling (so only the visible window is in the DOM) with a data-window strategy that also *evicts* far-scrolled-past data from memory, not just from the DOM.

```
@Component({
  selector: 'app-feed',
  standalone: true,
  imports: [ScrollingModule],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <cdk-virtual-scroll-viewport itemSize="220" (scrolledIndexChange)="onScroll($event)"
    class="feed">
      <app-post-card *cdkVirtualFor="let post of posts(); trackBy: trackByPostId"
      [post]="post" />
    </cdk-virtual-scroll-viewport>
  `,
})
export class FeedComponent {
  private feedStore = inject(FeedStore);
  posts = this.feedStore.visiblePosts; // windowed signal, e.g., keeps last 300 loaded

  trackByPostId = (_, p: Post) => p.id;

  onScroll(index: number) {
    if (index > this.posts().length - 10) {
      this.feedStore.loadMore();
    }
  }
}
```

```
// In the store: cap retained posts instead of growing an array forever
loadMore() {
  this._posts.update(existing => {
    const next = [...existing, ...this.fetchNextBatch()];
    return next.length > 300 ? next.slice(next.length - 300) : next; // evict oldest
  });
}
```

Also replace per-card `setInterval` "time ago" timers with a single shared ticking signal in a service that all cards read via `computed()`, so there's one timer instead of thousands.

Measurement: Memory snapshots after scrolling through 2,000+ posts should plateau rather than grow linearly; DOM node count stays bounded to the virtual-scroll window; the Performance trace's timer-related task cluster should shrink from thousands of per-second callbacks to one.

Interviewer intent: Tests whether the candidate thinks beyond DOM virtualization to also address *data* growth and per-item background work as compounding leak sources — a step beyond the basic virtual-scroll answer.

Scenario 13: Nested @for Loops Cause Exponential Slowdown on Filter

Situation: A permissions matrix UI renders `@for (role of roles(); track role.id)` containing a nested `@for (perm of permissions(); track perm.id)`, each cell computing `hasPermission(role, perm)` inline in the template. With 50 roles × 80 permissions (4,000 cells), typing in a filter box takes ~700ms to update the view.

Question: Why is this nested loop so slow, and what's the fix?

Answer:

Diagnosis: The Performance trace shows a huge "Evaluate script" block during each filter keystroke, dominated by repeated calls to `hasPermission`. Since `hasPermission(role, perm)` is called directly from the template, Angular invokes it on *every change detection cycle* for *every one of the 4,000 cells* — and if the function does a `.find()` or `.some()` scan over an array internally, that's effectively $O(\text{roles} \times \text{permissions} \times \text{scanSize})$ work repeated on every CD pass, not just once per actual data change.

Root cause: calling a non-memoized function directly in a template expression inside a doubly-nested loop. Angular templates invoke method calls on every CD cycle by design (no automatic memoization), so any moderately expensive function in a hot template path becomes a quadratic-plus bottleneck at scale.

Fix: precompute the permission matrix into a lookup structure via `computed()` whenever roles/permissions change, so the template only does $O(1)$ map lookups per cell.

```
@Component({
  selector: 'app-permission-matrix',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (role of filteredRoles(); track role.id) {
      <div class="row">
        @for (perm of permissions(); track perm.id) {
          <app-perm-cell [checked]="matrix()[role.id]?.[perm.id] ?? false" />
        }
      </div>
    }
  `,
})
export class PermissionMatrixComponent {
  roles = input.required<Role[]>();
  permissions = input.required<Permission[]>();
  filterText = signal('');

  filteredRoles = computed(() =>
    this.roles().filter(r =>
      r.name.toLowerCase().includes(this.filterText().toLowerCase()))
  );
}
```

```
// Computed once per roles/permissions change, NOT per CD cycle or per keystroke
matrix = computed(() => {
  const map: Record<string, Record<string, boolean>> = {};
  for (const role of this.roles()) {
    map[role.id] = {};
    for (const perm of this.permissions()) {
      map[role.id][perm.id] = role.permissionIds.includes(perm.id);
    }
  }
  return map;
});
}
```

Because `matrix` only recomputes when `roles/permissions` signals actually change — not when `filterText` changes — filtering only re-runs the cheap `filteredRoles` computed and re-renders a subset of rows via `@for`'s `trackBy`-based diffing, leaving the $O(1)$ `matrix()[id][id]` lookup untouched.

Measurement: Re-trace a filter keystroke — the long "Evaluate script" block should disappear, replaced by a small filtering pass over 50 roles; total keystroke-to-render time should drop from ~700ms to under 20ms.

Interviewer intent: Tests recognition that template method calls re-run every CD cycle and that memoizing derived data structures with `computed()` is the fix for expensive nested-loop template logic, not just adding `OnPush`.

Scenario 14: App Freezes for 3 Seconds When Exporting a Large Report to CSV

Situation: Clicking "Export to CSV" on a report with 100,000 rows freezes the entire UI (unresponsive to clicks, spinner doesn't even animate) for about 3 seconds before the download starts.

Question: Why does a client-side export freeze the whole UI, and how do you keep it responsive?

Answer:

Diagnosis: The Performance trace shows one massive synchronous long task (the entire 3-second block) on the main thread, matching the CSV-building function (likely a `.map().join(',')` style string-builder over 100,000 rows plus escaping logic). Because JavaScript is single-threaded and this runs synchronously, it blocks not just rendering but all event handling — hence the spinner (which itself needs a repaint) never animates.

Root cause: CPU-bound synchronous work performed entirely on the main thread with no yielding, blocking the event loop for the whole duration.

Fix, in order of increasing effort: chunk the work with yielding, or move it off the main thread entirely with a Web Worker.


```
// Simple fix: yield to the event loop periodically so paints/input can interleave
async function buildCsvChunked(rows: ReportRow[]): Promise<string> {
  const chunks: string[] = [];
  const CHUNK_SIZE = 2000;
  for (let i = 0; i < rows.length; i += CHUNK_SIZE) {
    const slice = rows.slice(i, i + CHUNK_SIZE);
    chunks.push(slice.map(rowToCsvLine).join('\n'));
    await new Promise(resolve => setTimeout(resolve, 0)); // yield
  }
  return chunks.join('\n');
}
```

```
// Better fix: Web worker, keeps main thread fully free
// export.worker.ts
addEventListener('message', ({ data }: MessageEvent<ReportRow[]>) => {
  const csv = data.map(rowToCsvLine).join('\n');
  postMessage(csv);
});

// component.ts
export class ReportComponent {
  exportToCsv() {
    const worker = new Worker(new URL('./export.worker', import.meta.url));
    worker.postMessage(this.reportRows());
    worker.onmessage = ({ data }) => {
      this.downloadCsv(data as string);
      worker.terminate();
    };
  }
}
```

The chunked/yielding approach is simpler and good enough for a few seconds of work; the worker approach is preferable if exports can be much larger or run frequently, since it guarantees zero main-thread impact regardless of dataset size.

Measurement: Re-trace the export click — instead of one 3-second block, the Performance panel should show either many small (<50ms) tasks interleaved with idle time (chunked version) or the work happening entirely on a separate worker thread track with the main thread staying responsive to input the whole time. Confirm by clicking a button during export — it should respond immediately instead of queuing behind the freeze.

Interviewer intent: Tests whether the candidate knows to look for CPU-bound synchronous blocking work as a distinct performance category from rendering/CD issues, and knows both the cheap (chunking) and correct (worker) fixes.

Scenario 15: Angular Material Data Table with Custom Cell Renderers Is Slow to Sort

Situation: A `mat-table` with 2,000 rows and several custom cell components (badges, avatars, computed status pills) takes ~1.1 seconds to re-sort when a user clicks a column header — noticeably worse than a plain table would.

Question: Why does sorting take over a second, and how do you speed it up?

Answer:

Diagnosis: A Performance trace on the sort click shows the majority of time not in the actual `Array.sort()` call (which finishes in a few ms) but in "Recalculate Style"/"Layout" and component re-render — every row's custom cell components (badge, avatar, status pill) are being destroyed and recreated because `MatTableDataSource`'s default sort creates a *new array with new object wrapper references* (or the row-tracking isn't stable), and if the cell components aren't `OnPush`, or the table doesn't provide a `trackBy`, Angular treats the whole re-sorted list as entirely new content.

Root cause: (1) missing `trackBy` on the table so identity is positional, not row-ID based, meaning every row after the sort is "new" from Angular's perspective at whatever position it lands in; (2) custom cell components not on `OnPush`, so each one does more re-render work than necessary even if reused.

Fix:

```
@Component({
  selector: 'app-user-table',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <table mat-table [dataSource]="dataSource" matSort [trackBy]="trackByUserId">
      <ng-container matColumnDef="status">
        <th mat-header-cell *matHeaderCellDef mat-sort-header>Status</th>
        <td mat-cell *matCellDef="let user">
          <app-status-pill [status]="user.status" />
        </td>
      </ng-container>
      <!-- other columns -->
    <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
    <tr mat-row *matRowDef="let row; columns: displayedColumns"></tr>
  </table>
  `,
})
export class UserTableComponent {
  trackByUserId = (index: number, row: User) => row.id;
}

@Component({
  selector: 'app-status-pill',
  changeDetection: ChangeDetectionStrategy.OnPush, // cheap to reuse, not recreate
  template: `<span class="pill" [class]="status()">{{ status() }}</span>`,
})
export class StatusPillComponent {
  status = input.required<string>();
}
```

Additionally, avoid a custom `sortingDataAccessor` that does expensive parsing (e.g., re-parsing a date string) on every comparison — precompute a sortable numeric/date field once when data loads instead of inside the comparator, since `Array.sort` calls the accessor $O(n \log n)$ times.

Measurement: Re-trace the sort click — the "Recalculate Style"/component-render portion should shrink drastically since `trackBy` lets Angular reuse existing `StatusPillComponent` (and similar) instances and DOM nodes, just reordering and patching bindings. Total sort-to-render time should drop from ~1.1s to under 150ms for 2,000 rows.

Interviewer intent: Tests transfer of the `trackBy` / OnPush lesson specifically into the Angular Material table context, which many candidates treat as a black box.

Scenario 16: Signal-Based Effect Runs in an Infinite Loop Under Load

Situation: A settings panel uses an `effect()` to sync a signal to `localStorage`. Under normal use it's fine, but QA reports that after a specific sequence of actions, the browser tab becomes 100% CPU-pegged and DevTools shows the console flooded with the same effect running thousands of times per second.

Question: What causes an Angular effect to run away like this, and how do you prevent it?

Answer:

Diagnosis: The Performance trace (or even just the Console with a log statement inside the effect) shows the same `effect()` callback firing continuously. Reading the effect's code reveals it both *reads* and *writes* the same signal (directly or transitively through a signal it also updates), creating a reactive feedback loop: the effect reads `settings()`, computes something, and calls `settings.set(...)` inside itself, which re-triggers the effect, which reads `settings()` again, and so on. Angular's effect scheduler will actually throw `NG0600` ("writing to signals is not allowed in a computed or effect...") in stricter cases, but if the write happens asynchronously (e.g., inside a `.then()` or `setTimeout` within the effect) that guard doesn't catch it, and you get a genuine infinite/near-infinite loop instead.

Root cause: mutating a signal from within its own effect's reactive dependency chain — a classic reactive-programming pitfall, here surfaced through `localStorage` sync logic that reads `settings()`, "normalizes" it, and writes the normalized version back to the *same* signal.

Fix: separate "source of truth" writes from "side-effect" reads. An effect that persists to `localStorage` should only *read*, never write back to the signal it depends on; any normalization should happen at the point of the original `.set()` / `.update()` call, not reactively.

```
// Bad: effect reads and writes the same signal it depends on
effect(() => {
  const current = this.settings();
  const normalized = normalize(current);
  if (JSON.stringify(normalized) !== JSON.stringify(current)) {
    this.settings.set(normalized); // <-- re-triggers this same effect
  }
  localStorage.setItem('settings', JSON.stringify(normalized));
});

// Good: effect is read-only (persistence side effect), normalization happens at the write
site
setSettings(raw: Settings) {
  this.settings.set(normalize(raw)); // normalize once, at the source
}
```

```
effect(() => {
  localStorage.setItem('settings', JSON.stringify(this.settings())); // pure read, no
  feedback loop
});
```

If a write-back is genuinely required (e.g., syncing a derived signal), use `untracked()` to read without establishing a dependency, or restructure as a `computed()` for the derived value instead of an effect with a manual write:

```
effect(() => {
  const current = this.settings();
  untracked(() => {
    // reads here don't create dependencies, but writes still should be avoided in effects
    generally
  });
});
```

Measurement: After the fix, the console/Performance trace should show the effect running exactly once per genuine `settings` change (e.g., once per user edit), not continuously. CPU usage in Task Manager should return to near-idle when the panel is left open without interaction.

Interviewer intent: Tests deep understanding of Angular Signals' reactive model and a real footgun (effects that write to their own dependencies), not just component-level CD tuning.

Scenario 17: Icon Font / SVG Sprite Causes Flash of Unstyled Icons and Reflow

Situation: A navigation sidebar with 25 icons shows a visible "flash" where icons pop in about 400ms after the rest of the layout, each pop-in shifting adjacent text slightly (small but frequent layout shifts, contributing 0.08 to CLS specifically from the sidebar).

Question: How do you diagnose and eliminate icon-related layout shift and flash?

Answer:

Diagnosis: The Network panel shows an icon font file (`icons.woff2`) or a sprite sheet loading well after initial paint, and the Rendering panel's layout-shift regions highlight each icon's bounding box as it "snaps" from a fallback/empty size to its real size. If using an icon font, elements with no glyph yet render as zero-width or use the fallback font metrics until the custom font loads (FOUT-like behavior specific to icon glyphs).

Root cause: icon containers have no reserved dimensions before the icon asset loads, so the browser lays out the row based on empty/fallback content and then reflows once the real icon renders.

Fix: reserve explicit space for every icon regardless of load state, and prefer inline SVG (via Angular's built-in support for `[innerHTML]` sanitization or, better, a sprite `<use>` reference) over icon fonts, since SVGs can be sized via CSS immediately without waiting on a font file.

```

@Component({
  selector: 'app-nav-icon',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <svg class="icon" width="20" height="20" aria-hidden="true">
      <use [attr.href]='"/assets/icons/sprite.svg#' + name()'></use>
    </svg>
  `,
  styles: [`.icon { display: inline-block; width: 20px; height: 20px; }`], // reserved
  space, no shift
})
export class NavIconComponent {
  name = input.required<string>();
}

```

Preload the sprite so it's available before first paint of the sidebar:

```

<link rel="preload" href="/assets/icons/sprite.svg" as="image" type="image/svg+xml" />

```

If icon fonts must be kept (e.g., a third-party library), apply `font-display: block` with a short block period plus explicit width/height on the containing element so at minimum the layout doesn't shift even if the glyph itself pops in slightly late.

Measurement: Re-run Lighthouse — the CLS contribution attributed to the sidebar should drop to ~0, and the Rendering panel should show no layout-shift regions around nav icons during load. Visually, on a throttled connection, icons should occupy their final box size immediately (possibly blank momentarily) rather than causing text reflow.

Interviewer intent: Extends the general CLS lesson (Scenario 8 was images) to a less obvious source — icon fonts/SVGs — testing whether the candidate generalizes "reserve space for async content" rather than memorizing one fix.

Scenario 18: `providedIn: 'root'` Service Re-Fetches Data on Every Component Instantiation

Situation: A `CurrentUserService` is injected into 15 different components across the app. The Network tab shows the `/api/me` endpoint being called 15 times on initial load (once per component), even though the user data is identical every time and rarely changes.

Question: Why is the same data being fetched repeatedly, and how do you fix the duplication?

Answer:

Diagnosis: Network waterfall shows 15 near-identical requests to `/api/me` firing within the same second. Reading `CurrentUserService`, each component calls `this.userService.fetchUser().subscribe(...)` independently in its own `ngOnInit`, and `fetchUser()` issues a *new* HTTP call every time it's invoked rather than caching/sharing an in-flight or completed result — a classic case of a singleton *service* not implying singleton *data*.

Root cause: no caching/multicasting on the underlying observable — each subscriber triggers its own independent HTTP request because plain `HttpClient.get()` returns a cold observable, and `providedIn: 'root'` only guarantees one service *instance*, not automatic request deduplication.

Fix: cache the result as a signal (or a `shareReplay(1)` 'd observable) computed once, so all consumers read the same resolved value instead of triggering their own fetch.

```
@Injectable({ providedIn: 'root' })
export class CurrentUserService {
  private http = inject(HttpClient);

  // Fetched once, shared by every injector of this service
  private user$ = this.http.get<User>('/api/me').pipe(shareReplay({ bufferSize: 1, refCount:
false }));
  user = toSignal(this.user$, { initialValue: null });

  // If manual refresh is ever needed:
  private refreshTrigger = signal(0);
}
```

```
// Consuming components simply read the signal – no fetch, no subscription management
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, template: `{{
userService.user()?.name }}` })
export class ProfileBadgeComponent {
  userService = inject(CurrentUserService);
}
```

For scenarios needing occasional refresh (e.g., after profile edit), model it as a signal-based resource (Angular's `resource()` / `rxResource()` API) keyed on a refresh trigger signal, so refetching is explicit and still shared across all consumers rather than per-component.

Measurement: Reload the app and check the Network tab — `/api/me` should fire exactly once regardless of how many components inject `CurrentUserService`. Confirm via the Angular DevTools "Injector Tree" that all 15 components resolve to the same service instance and the same underlying signal value without independent network activity.

Interviewer intent: Tests understanding that DI singleton scope doesn't automatically mean shared/cached HTTP results — a subtle but common source of redundant network waterfalls.

Scenario 19: Zone.js Overhead Shows Up as Constant Background CPU Usage

Situation: Chrome Task Manager shows the tab consuming 4-6% CPU continuously even when the app is completely idle (no user interaction, no polling configured). Profiling shows recurring short tasks tagged `zone.js` triggered by unrelated browser timer/animation activity (e.g., a CSS animation's `requestAnimationFrame`, or a third-party widget's internal polling).

Question: Why does an idle Angular app still consume CPU, and how do you address zone.js overhead specifically?

Answer:

Diagnosis: The Performance trace, filtered to `zone.js`-attributed tasks, shows `ApplicationRef.tick()` firing repeatedly even with no visible app activity — because Zone.js monkey-patches virtually all async browser APIs (`setTimeout`, `addEventListener`, `requestAnimationFrame`, Promise callbacks, XHR/fetch), *any* async activity anywhere on the page — including from a third-party widget or even an unrelated CSS/JS animation loop — triggers a full Angular change detection pass, whether or not anything Angular-relevant changed.

Root cause: this is Zone.js's fundamental design trade-off — global interception for automatic CD convenience, at the cost of running full-tree CD checks far more often than necessary, including for events Angular doesn't need to react to at all.

Fix: migrate (fully or partially) to zoneless change detection, which removes Zone.js's global monkey-patching and instead relies on Angular's signal-based fine-grained reactivity to schedule CD only when something Angular actually observes has changed.

```
// app.config.ts
import { provideExperimentalZonelessChangeDetection } from '@angular/core';
// (API name varies by exact Angular version; check current release notes for the finalized export)

export const appConfig: ApplicationConfig = {
  providers: [
    provideExperimentalZonelessChangeDetection(),
    provideRouter(routes),
  ],
};
```

This requires the codebase to be signal-driven (or to manually call `markForCheck()` /use `async` pipe with an observable-to-signal bridge) since Angular can no longer rely on Zone patching to know "something happened" — but the payoff is that unrelated third-party timers/animations no longer trigger Angular CD at all.

If a full zoneless migration isn't feasible yet, a partial mitigation is running noisy third-party widgets **outside Angular's zone** so their internal timers don't trigger app-wide CD:

```
export class ThirdPartyWidgetComponent {
  private ngZone = inject(NgZone);
  private el = inject(ElementRef);

  ngAfterViewInit() {
    this.ngZone.runOutsideAngular(() => {
      initThirdPartyWidget(this.el.nativeElement); // its internal setInterval/rAF won't trigger Angular CD
    });
  }
}
```

Measurement: Re-check Chrome Task Manager idle CPU usage — after `runOutsideAngular` wrapping (or a full zoneless migration), idle CPU should drop close to 0%, and the Performance trace should show `zone.js`-attributed tasks disappearing for events unrelated to actual Angular-observed state changes.

Interviewer intent: Tests awareness of Zone.js's global-patching cost as a distinct, architecture-level performance concern, and familiarity with zoneless Angular / `runOutsideAngular` as mitigations — a strong signal of staying current with the framework's direction.

Scenario 20: `@defer` Block Never Triggers Because It's Already in the Initial Viewport

Situation: A team added `@defer (on viewport)` around a heavy comments section to improve initial load, but Lighthouse's bundle report shows the comments chunk is still downloaded and executed immediately on page load — no deferral benefit was observed at all.

Question: Why didn't `@defer (on viewport)` actually defer anything here, and what's the fix?

Answer:

Diagnosis: Inspect the page layout — the comments section, due to a short article body above it, sits within the initial viewport on page load (no scrolling needed to see it). `@defer (on viewport)` uses an

`IntersectionObserver` to trigger loading as soon as the placeholder *enters* the viewport — if it's already in the viewport at load, the trigger fires essentially immediately, providing no meaningful deferral, which is actually correct/expected behavior for that trigger, just not what the team wanted.

Root cause: mismatch between the chosen `@defer` trigger and the actual layout/UX goal — `on viewport` is for "below the fold, load when scrolled to," not "load later regardless of position." The team needed a trigger based on idle time, interaction, or a custom condition instead.

Fix: choose a trigger that matches intent. If the goal is "don't block initial paint/TTI, but still load soon after," use `on idle` (loads once the browser is idle after initial render) possibly combined with a minimum placeholder duration for a smoother UX; if the goal is "only load when the user actually wants comments," gate it behind interaction.

```
@Component({ /* ... */ })
export class ArticleComponent {
  showComments = signal(false);
}
```

```
<!-- Option A: load once the browser is idle post-initial-render, regardless of scroll position -->
```

```
@defer (on idle) {
  <app-comments [articleId]="articleId()" />
} @placeholder {
  <div class="comments-skeleton"></div>
}
```

```
<!-- Option B: only load on explicit user interaction -->
```

```
@defer (on interaction) {
  <app-comments [articleId]="articleId()" />
} @placeholder {
  <button>Show comments</button>
}
```

```
<!-- Option C: combine triggers – whichever fires first -->
```



```
@defer (on idle; on interaction) {
  <app-comments [articleId]="articleId()" />
} @placeholder {
  <div class="comments-skeleton"></div>
}
```

Also verify with the Network panel's "Initiator" column that the comments chunk request is now correctly initiated by the idle/interaction callback rather than appearing in the initial document's request waterfall.

Measurement: Re-run Lighthouse — the comments chunk should no longer count toward the initial bundle/TBT contribution; the Network tab should show it loading after the `idle` callback (typically a second or more after `load`) or only after a genuine user click, and Time to Interactive for the initial view should improve correspondingly.

Interviewer intent: Tests precise understanding of each `@defer` trigger's actual semantics rather than treating `@defer` as a magic "make it lazy" annotation — a common shallow-knowledge trap.

Quick Revision Cheat Sheet

- **Default CD checks the whole tree** — `OnPush` + signals scope re-renders to only what actually changed; always pair `OnPush` with stable input references (avoid inline object/array literals or function calls in templates).
- **Large lists need virtualization, not just `track`** — use CDK `cdk-virtual-scroll-viewport` when DOM node count, not just diffing cost, is the bottleneck; always set a stable `track / trackBy` (business ID, not `$index`) to avoid destroy/recreate churn.
- **Bundle size is fixed with lazy routes + `@defer`**, not micro-optimizations — profile with `source-map-explorer / webpack-bundle-analyzer`, enforce budgets in `angular.json`, and match the `@defer` trigger (`on viewport`, `on idle`, `on interaction`, `prefetch`) to actual intent.
- **Input lag is fixed with debounce + memoized `computed()` + `OnPush`**, and Web Workers for genuinely CPU-heavy filtering/parsing so the main thread stays free.
- **Reference identity defeats `OnPush`** — new object/array literals recreated every CD pass silently disable `OnPush`'s benefit; split shared state signals into independent, narrowly-scoped signals instead of one big object.
- **Route transitions can be blocked by resolvers** — prefer navigating immediately and streaming data in with a loading/skeleton state, and preload likely-next chunks.
- **Third-party scripts should load `async / defer` and initialize on idle** or behind a facade/interaction gate — don't let vendor code compete with Angular bootstrap for the main thread.
- **CLS comes from unreserved space** — `NgOptimizedImage` for images, explicit sized containers for icons/SVGs/ads, and `priority / fetchpriority` for above-the-fold hero content.
- **Memory leaks are diagnosed via heap snapshot diffing** — look for detached DOM and growing subscriber/listener counts; fix with `takeUntilDestroyed()` / `DestroyRef.onDestroy()` for every subscription, listener, interval, and WebSocket.
- **Effects that write to signals they read create infinite reactive loops** — keep effects read-only for side effects (persistence, logging); normalize data at the write site, not reactively inside an effect.
- **DI singleton scope ≠ shared cached data** — cache HTTP results with `shareReplay / signal-backed`

resources so multiple components sharing a service don't each trigger independent network requests.

- **Zone.js triggers CD on *any* async browser event, even unrelated ones** — use `runOutsideAngular` for noisy third-party timers, or migrate to zoneless change detection for the cleanest fix.
- **Always close the loop with measurement** — re-run the same profiling tool (Performance trace, Lighthouse, Memory snapshot, Task Manager) after the fix and confirm the specific metric that motivated the investigation actually improved.

Created By - Durgesh Singh